

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1011	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	AKTCHAYA A	Reg. No.:	192424096
Date:	17/08/2026	Duration:	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Git & Docker Technical Assignment

1. Problem Analysis

- **Problem Statement**

The University Academic Management and Student Success Platform is a centralized system designed to manage student admissions, course registration, attendance, assessments, examinations, academic records, faculty activities, and student performance. The platform helps universities reduce manual work, improve communication, monitor academic progress, identify academically at-risk students, and generate useful academic reports.

- **Objectives**

- Manage student academic records.
- Manage course registration and examinations.
- Monitor attendance and academic performance.
- Manage faculty activities.
- Notify students, faculty, and advisors.
- Identify academically at-risk students.
- Generate academic reports.
- Support institutional planning and decision-making.

- **Functional Requirements**

- Student and faculty management.
- Admission management.
- Course and subject management.
- Course registration.
- Attendance management.
- Assessment and examination management.
- Academic performance monitoring.
- At-risk student identification.
- Notification management.
- Report generation.
- Role-based administration.

- **Expected Outcomes**

The system is expected to improve academic administration, reduce manual work, increase student success, improve faculty productivity, provide timely academic intervention, and support efficient university management.

2. Project Structure Design

The project repository is organized into separate folders and files so that the application is easy to develop, test, maintain, and deploy.

Project Structure

1. **app/**

- Contains the main application source code.
- Includes the core functionality of the University Academic Management Platform.

2. **app/main.py**

- Main entry point of the Flask application.
- Starts the web server and defines the main application routes.

3. **models/**

- Contains the data models used by the application.
- Represents entities such as students, faculty, courses, attendance, and examinations.

4. **routes/**

- Contains application routes or API endpoints.
- Handles operations such as student management, course registration, attendance, and examinations.

5. **services/**

- Contains the main business logic.
- Performs activities such as academic performance analysis and identifying at-risk students.

6. **tests/**

- Contains test files for checking application functionality.
- Helps verify that the different modules work correctly.

7. **data/**

- Stores sample or testing data.
- Can contain student, course, attendance, and academic performance datasets.

8. **docs/**

- Contains project documentation.
- Can include architecture diagrams, workflow diagrams, and technical documentation.

9. **README.md**

- Provides an overview of the project.
- Contains features, installation instructions, Git commands, and Docker usage instructions.

10. requirements.txt

- Contains the Python libraries required by the application.
- For example, Flask and other project dependencies.

11. Dockerfile

- Contains instructions for creating the application's Docker image.
- Defines the Python environment, dependencies, application files, port, and startup command.

12. docker-compose.yml

- Defines and manages multiple Docker services.
- Can be used to run the application and PostgreSQL database together.

13. .gitignore

- Specifies files and folders that should not be tracked by Git.
- Examples include virtual environments, cache files, and environment configuration files.

14. .env

- Stores environment-specific configuration values.
- Can contain database connection details and other configuration settings.
- Sensitive information should not be uploaded to GitHub.

3. Git Repository Initialization

• Commands

- mkdir university-academic-platform
- cd university-academic-platform
- git init
- git status
- git add .
- git commit -m "Initial project setup"
- git branch -M main
- git remote add origin <repository-url>
- git push -u origin main

• Important Git Files

- .git/ – Stores Git repository information.
- .gitignore – Specifies files that should not be tracked.
- README.md – Contains project documentation.

4. Git Branching Strategy

The project uses the following branching model:

- main
- development
 - feature-student
 - feature-course
 - feature-attendance
 - feature-examination
 - feature-risk-detection
- Main: Contains stable code.
- Development: Contains integrated development work.
- Feature branches: Used for individual modules.
- Release branch: Used to prepare a stable release.
- Hotfix branch: Used to fix critical production issues.

5. Version Control Workflow

- **The basic Git workflow is:**
 - git checkout development
 - git checkout -b feature-student
 - git add .
 - git commit -m "Add student management module"
 - git push -u origin feature-student
- **After development, the feature branch is merged into the development branch:**
 - git checkout development
 - git merge feature-student
 - git push origin development
 - Conflict Resolution
- **If a conflict occurs:**
 - git status
- **The conflicting file is edited manually to keep the correct changes. Then:**
 - git add .
 - git commit -m "Resolve merge conflict"
 - git push origin development

6. Commit History Analysis

- **The project commit history can be viewed using:**

- git log --oneline --graph --all
- Example:
- Add risk detection module
- Add examination management
- Add attendance management
- Add course registration
- Add student management
- Initial project setup

- **Meaningful commit messages such as:**

- Add student management module
- Add attendance calculation
- Implement examination results
- Add at-risk student detection
- Fix attendance calculation
- Update Docker configuration

7. Docker Environment Design

Docker is used to provide a consistent environment for the application. The main components that can be containerized are:

1. Application container – Runs the university platform.
2. Database container – Stores academic information.
3. Supporting services – Can handle notifications or other external functions.

Docker provides portability, dependency isolation, easier deployment, and scalability.

8. Dockerfile Development

- **Dockerfile**

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
```


CMD ["python", "app/main.py"]

- **Explanation**

- FROM – Selects the Python base image.
- WORKDIR – Sets the working directory.
- COPY – Copies project files into the container.
- RUN – Installs required dependencies.
- EXPOSE – Specifies the application port.
- CMD – Starts the application.

- **Build and Run**

- `docker build -t university-academic-platform .`
- `docker run -p 8000:8000 university-academic-platform`

9. Docker Compose Design

- **docker-compose.yml**

services:

app:

build: .

ports:

- "8000:8000"

environment:

DATABASE_URL: postgresql://university:password@db:5432/universitydb

depends_on:

- db

db:

image: postgres:16

environment:

POSTGRES_USER: university

POSTGRES_PASSWORD: password

POSTGRES_DB: universitydb

volumes:

- university_data:/var/lib/postgresql/data

volumes:

university_data:

- **Components**

- app: Runs the university application.
- db: Runs the PostgreSQL database.
- environment: Provides database configuration.
- depends_on: Connects the application dependency with the database.
- volumes: Provides persistent database storage.
- network: Allows containers to communicate with each other.

- **Start the services using:**

`docker compose up --build`

- **Stop them using:**

`docker compose down`

10. Container Deployment and Testing

- **The application can be deployed using:**

`docker compose up --build`

- **Check running containers:**

`docker ps`

- **The application can be accessed through:**

`http://localhost:8000`

- **Testing**

The following functions can be tested:

Test	Expected Result
Student registration	Student is registered
Course registration	Course is successfully registered
Attendance	Attendance is recorded
Assessment	Marks are stored
Examination	Results are displayed
Risk detection	At-risk students are identified
Reports	Academic reports are generated

11. Benefits of Git and Docker

- **Git Benefits**

- Supports team collaboration.

- Maintains complete version history.
- Provides branching and merging.
- Helps resolve conflicts.
- Supports code review.
- Makes project changes traceable.
- **Docker Benefits**
 - Provides a consistent environment.
 - Simplifies deployment.
 - Reduces dependency problems.
 - Improves portability.
 - Supports scalability.
 - Makes application maintenance easier.

12. Challenges and Improvements

- **Challenges**
 - Git merge conflicts may occur during team development.
 - Managing multiple branches can become difficult.
 - Different dependency versions may cause errors.
 - Database configuration may vary between systems.
 - Handling thousands of users requires scalability.
 - Academic data requires strong security.
- **Improvements**
 - Follow a standard Git branching strategy.
 - Use meaningful commit messages.
 - Use Docker for consistent environments.
 - Store configuration using environment variables.
 - Implement authentication and role-based access.
 - Add automated testing and CI/CD.
 - Use cloud deployment and container scaling.
 - Add monitoring and backup mechanisms.

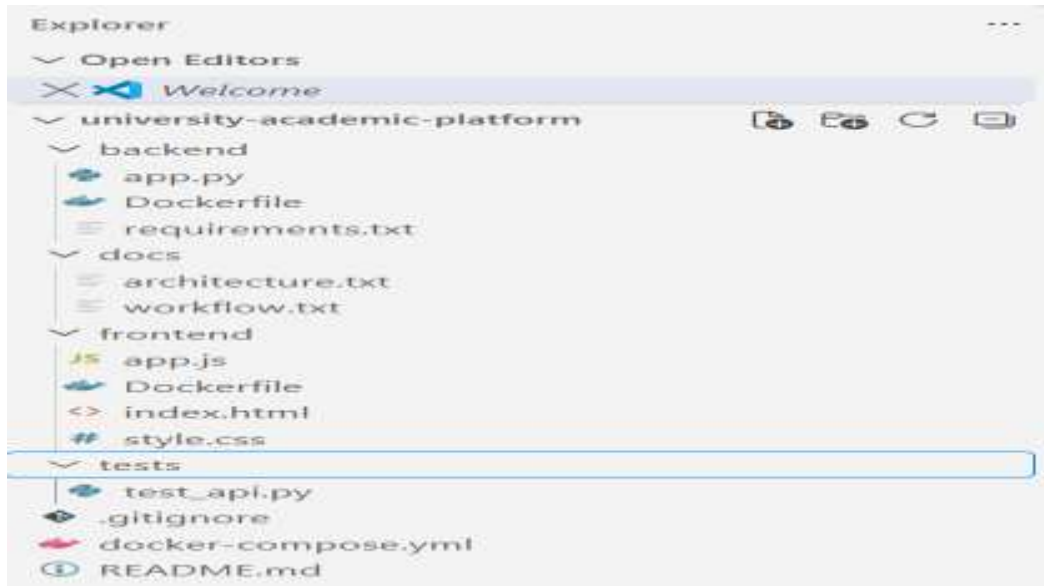
Conclusion

The University Academic Management and Student Success Platform provides a centralized solution for managing university academic activities. Git supports version control, collaboration, branching, and change management, while Docker provides a portable and consistent deployment

environment. The combination of Git and Docker improves the platform's collaboration, maintainability, portability, reliability, and scalability, making it suitable for managing academic operations across multiple departments and thousands of students.

Screenshots:

- **Project Structure**



- **Docker Image Build**

```
PS C:\Users\Hp\OneDrive\Desktop\university-academic-platform> docker compose build
#1 [internal] load local bake definitions
#1 reading from stdin 1.20kB done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 205B 0.0s done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 94B 0.0s done
#3 DONE 0.0s

#4 [frontend internal] load metadata for docker.io/library/nginx:alpine
#4 ...

#5 [auth] library/nginx:pull token for registry-1.docker.io
#5 DONE 0.0s

#6 [backend internal] load metadata for docker.io/library/python:3.11-slim
#6 ...

#7 [auth] library/python:pull token for registry-1.docker.io
#7 DONE 0.0s

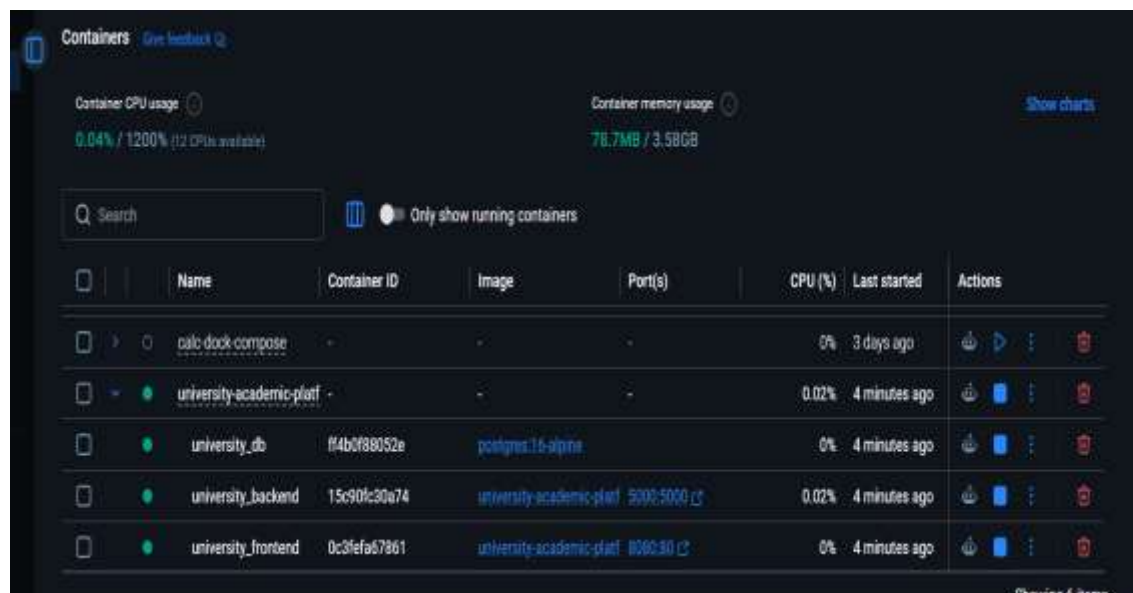
#6 [backend internal] load metadata for docker.io/library/python:3.11-slim
#6 ...

#4 [frontend internal] load metadata for docker.io/library/nginx:alpine
#4 DONE 5.8s
```

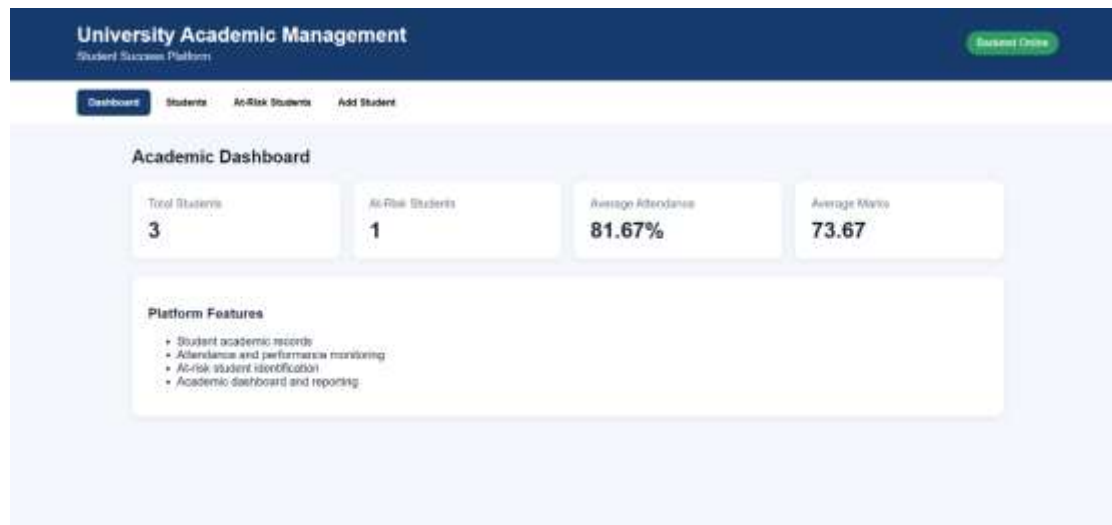
- **Docker Compose Startup**

```
PS C:\Users\Hp\OneDrive\Desktop\university-academic-platform> docker compose up
[+] up 18/18
✓ Image postgres:16-alpine Pulled 115.8s
✓ Network university-academic-platform_default Created 0.1s
✓ Volume university-academic-platform_university_db Created 0.0s
✓ Container university_db Created 0.3s
✓ Container university_backend Created 0.1s
✓ Container university_frontend Created 0.1s
Attaching to university_backend, university_db, university_frontend
Container university_db Waiting
university_db | The files belonging to this database system will be owned by user "postgres".
university_db | This user must also own the server process.
university_db |
university_db | The database cluster will be initialized with locale "en_US.utf8".
university_db | The default database encoding has accordingly been set to "UTF8".
university_db | The default text search configuration will be set to "english".
university_db |
university_db | Data page checksums are disabled.
university_db |
university_db | fixing permissions on existing directory /var/lib/postgresql/data ... ok
university_db | creating subdirectories ... ok
university_db | selecting dynamic shared memory implementation ... posix
university_db | selecting default max_connections ... 100
university_db | selecting default shared_buffers ... 128MB
university_db | selecting default time zone ... UTC
university_db | creating configuration files ... ok
university_db | running bootstrap script ... ok
university_db | sh: locale: not found
university_db | 2026-08-17 08:38:12.467 UTC [34] WARNING: no usable system locales were found
university_db | performing post-bootstrap initialization ... ok
university_db | syncing data to disk ... ok
university_db |
```

- **Docker Desktop Containers**



- Application Dashboard



- Student Records

The screenshot shows the 'Student Records' page. The header and navigation bar are consistent with the dashboard. The main content area displays a table of student records with the following data:

ID	Name	Department	Email	Attendance	Marks	Action
1	Arun Kumar	AI & DS	arun@university.edu	85.00%	78.00	<button>Delete</button>
2	Priya Sharma	Computer Science	priya@university.edu	68.00%	55.00	<button>Delete</button>
3	Rahul Raj	Information Technology	rahul@university.edu	92.00%	88.00	<button>Delete</button>

- At-Risk Students

The screenshot shows the 'Academically At-Risk Students' page. The header and navigation bar are consistent with the dashboard, with 'At-Risk Students' selected. The main content area includes a definition: 'A student is tagged when attendance is below 75% or marks are below 50.' Below this, a table lists the at-risk student:

Priya Sharma	Computer Science	Attendance: 68.00%	Marks: 55.00
--------------	------------------	--------------------	--------------

- Adding Students successfully

University Academic Management
Student Success Platform

Backend Online

Dashboard Students At-Risk Students Add Student

Add Student

AKASHA

Black-AUS

akash201@gmail.com

BS

93

Add Student

Student added successfully.

- Backend API Health

```

Pretty-print ☐

{"service":"university-backend","status":"healthy"}

```

- Pushing the project into the Github repository

